

RefactorTactics — PRD, roadmap e percorso didattico per Unreal Engine

Executive summary

RefactorTactics è un gioco tattico competitivo a turni simultanei, ispirato alla chiarezza decisionale di Atlas Reactor ma costruito attorno a tre elementi distintivi:

Pilastro	Promessa al giocatore
Mappa come sistema di gioco	Terreni, quote, collegamenti verticali, pericoli e oggetti modificano movimento, tiro e abilità
Pianificazione collaborativa privata	Gli alleati vedono in tempo reale il piano della squadra; i nemici non ricevono questi dati
Personaggi modulari	Ogni eroe conserva una forte identità, ma può essere configurato attraverso specializzazioni, modifiche alle abilità e gadget

Il progetto viene concepito anche come **percorso per imparare Unreal Engine** partendo da una buona conoscenza di C# e da poca esperienza in C++. La strategia raccomandata è utilizzare:

- **C++ Unreal** per il modello tattico, il networking, il pathfinding e i sistemi che richiedono testabilità;
- **Blueprint** per composizione, prototipazione, animazioni, effetti e configurazione dei contenuti;
- **Data Assets, Gameplay Tags e JSON** per personaggi, abilità, terreni e mod;
- **Lua o un altro linguaggio sandboxato** soltanto nella fase finale dedicata alle mod;
- **C# come laboratorio opzionale**, non come dipendenza fondamentale del prodotto.

Unreal Engine 5.8 è stato pubblicato il 17 giugno 2026 e l'hotfix 5.8.1 è arrivato il 28 luglio 2026. Per il progetto è quindi sensato adottare **UE 5.8.1 come baseline iniziale**, bloccando poi la versione del motore durante ogni milestone importante. ¹

Assunzioni progettuali

Gli elementi seguenti non sono stati specificati e vengono quindi trattati come ipotesi modificabili:

Elemento	Assunzione di lavoro
Budget	Non specificato
Dimensione del team	Scenario A: sviluppatore singolo part-time; scenario B: piccolo team da 3-5 FTE
Piattaforme	PC Windows iniziale; console non incluse nelle stime
Modalità principale	Competitivo 4 contro 4

Elemento	Assunzione di lavoro
Durata partita	Circa 20–35 minuti
Modello commerciale	Non specificato
Backend online	Non specificato; inizialmente sessioni dirette o servizio minimale
Live service	Non incluso nell'MVP
User-generated content pubblico	Previsto soltanto dopo la stabilizzazione del gioco

Pacchetto pronto per la stampa

Il report è stato impaginato per **A4**, con grafici a 300 DPI e versioni SVG vettoriali.

- [Scarica il PDF A4 stampabile](#)
- [Apri la versione HTML stampabile](#)
- [Scarica il kit completo ZIP con report, grafici e starter code](#)

Product requirements document

Visione

RefactorTactics deve offrire combattimenti tattici nei quali la qualità di una decisione non dipende soltanto dall'abilità scelta, ma dalla capacità di:

1. prevedere contemporaneamente le intenzioni avversarie;
2. coordinarsi con la propria squadra;
3. sfruttare la struttura tridimensionale e gli stati della mappa;
4. costruire un personaggio adatto alla strategia senza ottenere vantaggi permanenti ingiusti.

La promessa sintetica è:

Pianifica insieme, sfrutta la mappa, sorprendi l'avversario.

Pubblico di riferimento

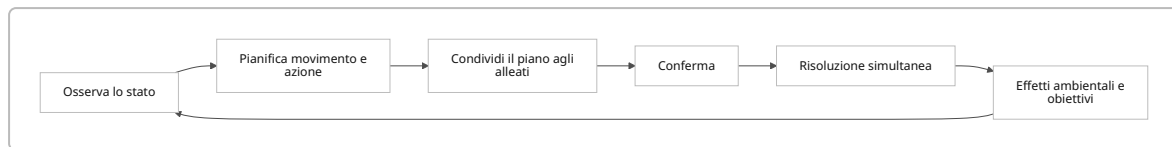
Il target primario comprende giocatori interessati a strategici tattici, hero game e combattimenti competitivi basati sulla previsione anziché sui riflessi. Il target secondario comprende creatori di mappe, modder e community interessate a modalità personalizzate.

Il prodotto dovrà essere leggibile anche senza chat vocale. Percorsi, aree d'effetto, intenzioni e conflitti tra alleati devono essere rappresentati direttamente sulla mappa.

Core loop della partita

Core loop del turno

Il ciclo fondamentale di un turno è:



Durante la pianificazione, ciascun giocatore:

- seleziona un percorso;
- seleziona un'azione;
- definisce bersaglio, direzione o area;
- può aggiungere un'etichetta strategica;
- vede i piani aggiornati dei compagni;
- riceve avvisi sui possibili conflitti;
- conferma quando è pronto.

Il server chiude la fase allo scadere del timer o quando tutti i giocatori hanno confermato. La risoluzione avviene secondo fasi prestabilite e deterministiche.

Struttura della risoluzione

Una prima versione gestibile può utilizzare la seguente sequenza:

Fase	Contenuto
Pre-turno	Trigger, stati, abilità preparate
Movimento	Movimento normale, dash, teletrasporti e collisioni
Reazioni	Overwatch, guardia, intercettazioni
Azioni principali	Attacchi, cure, abilità e interazioni
Impatti ambientali	Fuoco, elettricità, acqua, gas, crolli
Obiettivi	Controllo aree, punteggio, condizioni di vittoria
Cleanup	Cooldown, durata stati, rimozione entità

Non è necessario simulare fisicamente ogni dettaglio. Il sistema autorevole produce un **event log**; animazioni ed effetti visualizzano gli eventi senza ridefinirne il risultato.

Modalità iniziale

L'MVP dovrebbe includere una sola modalità:

Controllo tattico

Due squadre competono per uno o più punti di controllo. Le eliminazioni danno un vantaggio temporaneo, ma non devono essere l'unico modo per vincere. Questo permette alla mappa, al movimento e alle abilità di controllo di avere un ruolo centrale.

Una partita può terminare quando:

- una squadra raggiunge il punteggio obiettivo;
- termina il numero massimo di turni;
- una squadra non ha più unità recuperabili;
- una condizione specifica della mappa viene completata.

Priorità delle feature

Priorità	Feature	Contenuto	Criterio di completamento
P0	Turni simultanei	Planning, commit, risoluzione, aftermath	Stesso input e stesso seed producono lo stesso event log
P0	Griglia multilivello	Celle su più quote e collegamenti verticali	Unità in grado di raggiungere correttamente piani diversi
P0	Pathfinding semantico	A* con cost provider e requisiti	Percorsi differenti per unità con caratteristiche differenti
P0	LOS separata	Visibilità, cover e traiettorie indipendenti dal movimento	Preview e risultato server coincidono
P0	Planning privato	Intenti visibili solo agli alleati	Nessun payload del piano presente sul client avversario
P1	Ambiente interattivo	Terreni, hazard, oggetti, trasformazioni	Almeno cinque interazioni combinabili
P1	Personaggi modulari	Kit, specializzazione, augment e gadget	Due build dello stesso eroe cambiano strategia, non potenza assoluta
P1	Progressione orizzontale	Sblocchi non competitivamente superiori	Nessun bonus permanente a danno, salute o velocità in ranked
P1	IA di test	Bot semplici per scenario e QA	Possono completare un match senza bloccare il turno
P2	Editor di mappe	Posizionamento tile, connessioni e validazione	Una mappa completa creabile senza modificare il codice
P2	Data mod	JSON/Data Assets esterni	Nuovo terreno aggiunto senza ricompilare il core
P2	Script mod	API sandboxata e versionata	Mod di esempio distribuita senza codice nativo
P3	Workshop o portale mod	Pubblicazione e download	Dipende da piattaforma e modello commerciale

Mappa multilivello

La mappa non deve essere trattata come una semplice matrice bidimensionale con una coordinata Z aggiunta. Il modello corretto è un **grafo tattico semantico**.

Ogni cella è identificata da:

```
USTRUCT(BlueprintType)
struct FTileId
{
    GENERATED_BODY()

    UPROPERTY(EditAnywhere, BlueprintReadWrite)
    int32 X = 0;

    UPROPERTY(EditAnywhere, BlueprintReadWrite)
    int32 Y = 0;

    UPROPERTY(EditAnywhere, BlueprintReadWrite)
    int32 Layer = 0;
};
```

Una cella può contenere:

Categoria	Esempi
Geometria	Altezza, inclinazione, volume occupato
Movimento	Costo, blocco, capacità richiesta
Superficie	Acqua, ghiaccio, metallo, fango, vegetazione
Protezione	Cover parziale, cover totale, parete distruttibile
Visibilità	Fumo, oscurità, riflessi, occlusione
Pericolo	Fuoco, elettricità, veleno, radiazione
Interazione	Console, porta, ascensore, ponte, valvola
Stato temporaneo	Congelata, allagata, incendiata, destabilizzata
Occupazione	Personaggio, oggetto, ostacolo dinamico
Trigger	Entrata, uscita, inizio turno, fine turno, impatto

I collegamenti verticali sono archi del grafo:

```
Cella A, piano terreno
|
├─ scala: costo 20, requisito Movement.Climb
├─ ascensore: costo 10, requisito State.Powered
├─ salto: costo 15, requisito Ability.Jump
└─ portale: costo 5, requisito State.PortalActive
|
Cella B, piano superiore
```

Questo modello permette di aggiungere collegamenti che non corrispondono a una normale adiacenza geometrica.

Pathfinding semantico

A* rimane una base appropriata perché ricerca un percorso minimo combinando il costo già accumulato con una stima euristica del costo rimanente. La formulazione classica risale al lavoro di Hart, Nilsson e Raphael. ²

Il costo di attraversamento consigliato è:

```
Costo totale =
    costo dell'arco
+ costo base della cella
+ differenza di quota
+ modificatore del terreno
+ modificatore del personaggio
+ modificatore ambientale
+ modificatore temporaneo delle abilità
```

Il pathfinder non dovrebbe conoscere direttamente concetti come "lava", "robot" o "ghiaccio". Deve interrogare provider indipendenti:

```
class ITraversalCostProvider
{
public:
    virtual int32 GetAdditionalCost(
        const FTreeNode& From,
        const FTreeNode& To,
        const FUnitTraversalContext& Unit) const = 0;

    virtual bool CanTraverse(
        const FTreeNode& From,
        const FTreeNode& To,
        const FUnitTraversalContext& Unit) const = 0;
};
```

Provider previsti:

Provider	Responsabilità
TerrainCostProvider	Acqua, fango, ghiaccio, detriti
HeightCostProvider	Salita, discesa, caduta
HazardCostProvider	Fuoco, veleno, elettricità
UnitTraitProvider	Volo, isolamento, anfibio, pesante
AbilityCostProvider	Buff, debuff e campi temporanei

Provider	Responsabilità
OccupancyProvider	Unità, ostacoli e prenotazioni
InteractionProvider	Porte, ascensori e oggetti attivabili

Per il prototipo è preferibile utilizzare costi interi, ordinamenti stabili e una euristica semplice. Questo rende i test e il determinismo più facili.

Quando la mappa cambia, si incrementa una `GraphRevision`. Le cache conservano la revisione con cui sono state prodotte e vengono invalidate soltanto per le regioni interessate.

Algoritmi come *D Lite* o *Anytime Dynamic A* diventano interessanti quando i costi degli archi cambiano frequentemente e il costo del ricalcolo completo diventa misurabile. AD* combina ricerca anytime e riparazione incrementale, migliorando una soluzione quando è disponibile più tempo e riutilizzando il lavoro quando il grafo cambia. Non è però necessario introdurlo prima di avere dati di profiling reali.

3

Line of sight, cover e proiettili

LOS non deve essere incorporata nell'A*. Movimento e tiro sono query differenti.

Semantic Grid

- └─ Movement Query
- └─ Visibility Query
- └─ Cover Query
- └─ Projectile Query
- └─ Area Query

Una query LOS riceve:

```
FLineOfSightResult QueryLineOfSight(
    const FTileId& Source,
    const FTileId& Target,
    const FLineOfSightContext& Context);
```

E restituisce:

```
USTRUCT(BlueprintType)
struct FLineOfSightResult
{
    GENERATED_BODY()

    UPROPERTY(BlueprintReadOnly)
    bool bVisible = false;

    UPROPERTY(BlueprintReadOnly)
    float CoverFraction = 0.0f;
```

```
UPROPERTY(BlueprintReadOnly)
TArray<FTileId> TraversedTiles;
};
```

Le tracce fisiche di Unreal possono essere usate per validare geometria e occlusione, ma il risultato tattico deve dipendere da regole esplicite. Una ringhiera visivamente complessa, per esempio, dovrebbe avere un valore di cover definito e non produrre risultati casuali in base a piccoli dettagli della mesh.

Sistema dei personaggi

Ogni personaggio è composto da sei livelli:

```
Personaggio
├─ Identità base
├─ Kit caratteristico
├─ Specializzazione
├─ Augment delle abilità
├─ Gadget
└─ Tratti e affinità
```

L'identità base comprende silhouette, statistiche fondamentali, animazioni, ruolo narrativo e una o due meccaniche non sostituibili.

La specializzazione modifica il modo di utilizzare il kit:

Eroe di esempio	Specializzazione	Effetto
Echo	Ricognitore	Sensori, mobilità e rivelazione
Echo	Sabotatore	Mine, controllo porte e dispositivi
Echo	Tiratore	Posizionamento, altezza e linee di tiro

Gli augment cambiano la funzione di un'abilità:

```
Granata base
├─ Fragmentation: danno immediato
├─ Smoke: oscura la visibilità
├─ Shock: elettrifica superfici conduttive
└─ Cluster: crea tre mine più piccole
```

Il sistema deve evitare una combinazione completamente libera di ogni modulo. È preferibile utilizzare:

- slot limitati;
- moduli incompatibili;
- un budget di complessità;

- prerequisiti di specializzazione;
- tag di esclusione.

Progressione non distruttiva per il PvP

La progressione non deve assegnare incrementi permanenti come:

+10% danno
+20 punti salute
+1 movimento

Gli sblocchi possono invece includere:

Categoria	Contenuto
Opzioni laterali	Specializzazioni, augment e gadget equivalenti
Preset	Loadout salvati e configurazioni per mappa
Mastery	Titoli, badge, statistiche e sfide
Cosmetici	Skin, VFX, animazioni, emote e voice line
Allenamento	Scenari, replay annotati e strumenti di analisi
Narrazione	Lore, dialoghi e contenuti personali

In modalità classificata, tutte le opzioni gameplay possono essere disponibili fin dall'inizio oppure normalizzate. La progressione mantiene così motivazione e identità senza trasformarsi in un vantaggio competitivo cumulativo.

Pianificazione condivisa tra alleati

Durante la fase di planning, ogni giocatore vede per gli alleati:

- percorso;
- destinazione;
- posizione fantasma;
- direzione finale;
- bersaglio;
- area d'effetto;
- abilità prevista;
- stato pronto;
- intent label;
- eventuali conflitti.

Esempi di intent label:

Focus
Protezione
Scout

Trappola
Controllo
Ritirata
Attesa

Il sistema segnala, senza necessariamente bloccare:

Due alleati prevedono di terminare sulla stessa cella.
Il tiro attraversa una posizione alleata prevista.
La copertura potrebbe essere distrutta prima dell'azione.
Il bersaglio potrebbe uscire dalla linea di tiro.
L'abilità dipende da un ascensore non ancora attivato.

Quando un alleato modifica il piano, la visualizzazione deve aggiornarsi rapidamente, ma senza inviare un RPC a ogni singolo movimento del mouse. Una frequenza di circa 5-10 aggiornamenti al secondo, con invio soltanto in caso di cambiamento significativo, è una base da validare durante i test.

Privacy del planning

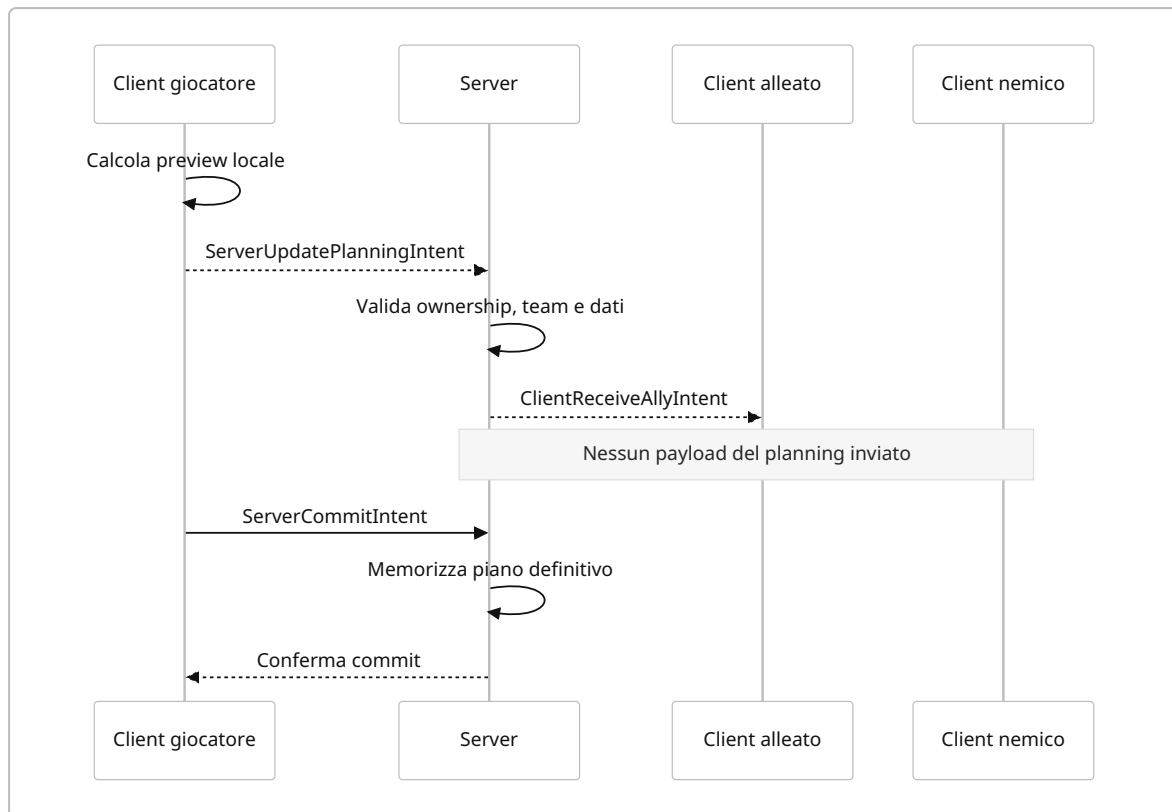
La privacy è un requisito di rete, non un'opzione grafica.

Il client nemico non deve ricevere:

- percorso pianificato;
- abilità selezionata;
- cella bersaglio;
- stato di conferma;
- etichetta d'intento;
- simulazione del risultato.

Nascondere un widget o un actor già replicato non è sufficiente. Unreal determina replicazione e rilevanza per connessione, supporta actor proprietari e condizioni come `COND_OnlyOwner`; queste primitive possono essere utilizzate per costruire una distribuzione dei dati limitata al team. ⁴

Flusso raccomandato:



Durante l'editing:

```

UFUNCTION(Server, Unreliable)
void ServerUpdatePlanningIntent(const FPlanningIntent& Intent);
  
```

Durante la conferma:

```

UFUNCTION(Server, Reliable)
void ServerCommitPlanningIntent(const FPlanningIntent& Intent);
  
```

Il server inoltra l'anteprima mediante Client RPC soltanto ai `PlayerController` appartenenti allo stesso team:

```

UFUNCTION(Client, Unreliable)
void ClientReceiveAllyIntent(
    int32 AllyPlayerId,
    const FPlanningIntent& Intent);
  
```

Il server deve validare:

- proprietà dell'unità;
- appartenenza al team;
- sequence number;
- frequenza degli aggiornamenti;

- disponibilità dell'abilità;
- compatibilità del percorso;
- dimensione massima del payload;
- contenuto dei tag;
- stato della fase di gioco.

Per l'MVP è preferibile utilizzare il sistema di replicazione stabile. Iris continua a essere indicato come sperimentale nella documentazione di UE 5.8, quindi non dovrebbe diventare una dipendenza critica prima di una spike separata. ⁵

Supporto alle mod

Il supporto alle mod è l'ultima grande feature, ma l'architettura deve prepararla fin dall'inizio.

La strategia è divisa in livelli:

Livello	Cosa può modificare	Tecnologia
Data mod	Terreni, abilità, effetti, statistiche, regole	JSON, Data Assets, Gameplay Tags
Content mod	Mappe, mesh, materiali, audio, VFX	Unreal Editor, cooking, pak e chunk
Script mod	Trigger, regole e comportamenti	Lua sandboxata
Native mod	Codice C++ arbitrario	Non ammesso nel catalogo pubblico

I Gameplay Tags forniscono etichette gerarchiche utili a descrivere abilità, terreni, requisiti e stati. I Primary Data Assets e l'Asset Manager offrono un sistema nativo per identificare, caricare e organizzare asset; cooking, chunking e pak possono essere utilizzati per separare contenuti aggiuntivi. ⁶

Esempio di manifest:

```
{
  "schemaVersion": 1,
  "modId": "com.example.electrified-water",
  "name": "Electrified Water",
  "version": "0.1.0",
  "gameApi": ">=0.9 <1.0",
  "author": "Example",
  "content": [
    "terrain/electrified_water.json"
  ],
  "scripts": [
    "Scripts/electrified_water.lua"
  ],
  "permissions": [
    "map.read",
    "effects.apply"
  ],
}
```

```

    "dependencies": []
}

```

Esempio di terreno:

```

{
  "id": "Terrain.ElectrifiedWater",
  "displayName": "Acqua elettrificata",
  "tags": [
    "Terrain.Liquid",
    "Hazard.Electric"
  ],
  "baseMoveCost": 18,
  "blocksMovement": false,
  "blocksVision": false,
  "onEnterEffects": [
    {
      "effect": "Status.Shocked",
      "magnitude": 1,
      "durationTurns": 1
    }
  ],
  "interactions": [
    {
      "action": "Ability.Freeze",
      "replaceWith": "Terrain.Ice"
    }
  ]
}

```

L'API di scripting non dovrebbe esporre liberamente `UObject`, filesystem o rete. Deve fornire handle e capability specifiche:

```

function mod.on_tile_enter(ctx)
  if ctx.tile:has_tag("Hazard.Electric")
    and not ctx.unit:has_tag("Trait.Insulated") then

      ctx.effects:apply(
        ctx.unit,
        "Status.Shocked",
        1,
        1
      )
    end
  end
end

```

Permessi possibili:

```

map.read
map.modify
unit.read
effects.apply
spawn.approved
ui.panel
storage.mod

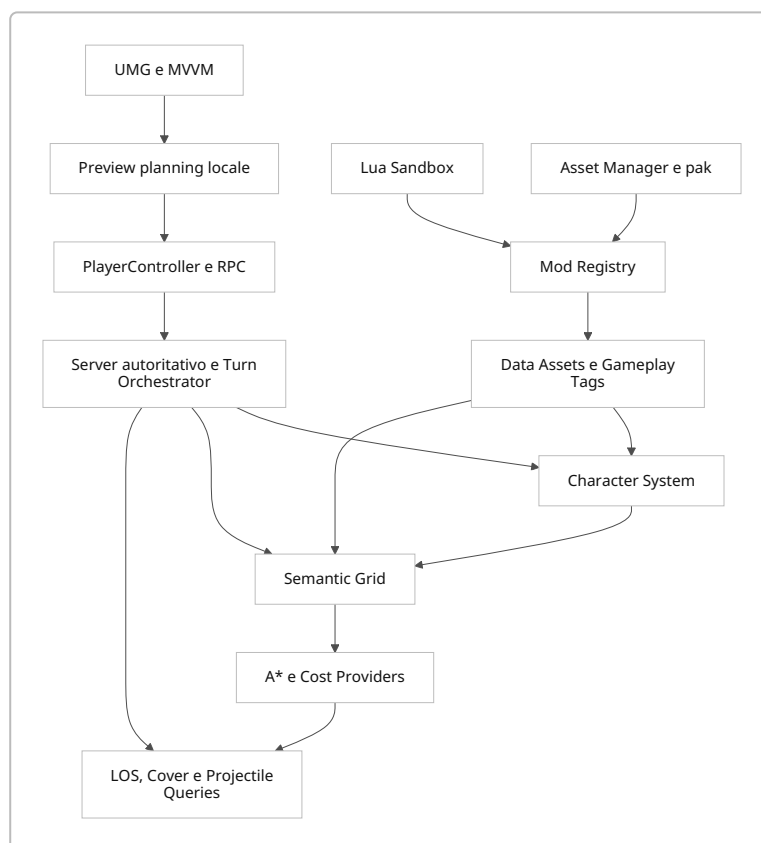
```

Ogni script deve avere limiti di tempo, memoria e numero di operazioni per turno. Nel multiplayer, server e client devono condividere lo stesso manifest, la stessa versione API e gli stessi hash per le mod che influenzano il gameplay.

Architettura e scelte tecnologiche

Architettura logica

Architettura logica



Responsabilità dei moduli

Modulo	Responsabilità
RefactorTacticsCore	Tipi di base, tag, ID, event log
RefactorTacticsMap	Tile graph, layer, archi e revisioni

Modulo	Responsabilità
RefactorTacticsPath	A*, provider, query di raggiungibilità
RefactorTacticsCombat	Abilità, danni, stati, LOS e cover
RefactorTacticsTurn	Planning, commit, resolve e aftermath
RefactorTacticsNet	RPC, validazione, privacy team e replay
RefactorTacticsUI	HUD, planning preview, notifiche e accessibilità
RefactorTacticsContent	Data Assets di personaggi, terreni e abilità
RefactorTacticsMod	Registry, manifest, JSON, sandbox e packaging
RefactorTacticsEditor	Tool per mappe, validazione e mod kit

Unreal organizza il gameplay attorno a classi come `Actor`, `Pawn`, `Controller`, `GameMode`, `GameState` e `PlayerState`; per chi proviene da Unity e C#, la documentazione ufficiale fornisce una corrispondenza tra `GameObject/MonoBehaviour` e `Actor/Component`, oltre al rapporto fra Blueprint e C++. ⁷

Blueprints, C++ e C

Tecnologia	Uso raccomandato	Vantaggi	Limiti
Blueprint	UI, animazioni, effetti, configurazione, prototipi	Feedback rapido, visuale, integrato con l'Editor	Grafi grandi difficili da mantenere e revisionare
C++ Unreal	Core tattico, rete, pathfinding, test, editor tool	Supporto nativo, performance, reflection e packaging	Sintassi e build più complesse rispetto a C#
C# con UnrealCLR	Spike o laboratorio isolato	.NET, C# moderno, debugger managed	Plugin di terze parti; compatibilità con UE 5.8 da verificare
Lua con UnLua	Mod scripting e iterazione controllata	Dinamico, accesso tramite reflection	Plugin community; necessaria sandbox e versione bloccata
AngelScript Hazelight	Possibile gameplay scripting avanzato	Esperienza d'uso in produzioni Hazelight	Richiede modifiche dirette al source dell'engine
MonoUE	Studio storico	Approccio C#/F#	Progetto e workflow non adatti come base moderna
VaRest	Non raccomandato	Accesso REST semplice da Blueprint	Repository archiviato e baseline engine datata

UnrealCLR dichiara integrazione .NET 6, C# 10, caricamento dinamico degli assembly e supporto desktop, ma la documentazione del repository usa prerequisiti generici e non garantisce

automaticamente la compatibilità con la versione 5.8. La scelta corretta è una spike time-boxed e non l'adozione immediata nel core. ⁸

MonoUE è rimasto a lungo un progetto sperimentale con workflow e limitazioni non adatti a una nuova produzione. ⁹

UnLua dichiara supporto a diverse versioni UE 4 e 5 e a più piattaforme, ma il suo issue tracker mostra che la compatibilità con nuove release deve essere verificata e fissata per ogni progetto. ¹⁰

L'integrazione AngelScript di Hazelight è attivamente documentata ed è stata usata nei giochi dello studio, ma richiede modifiche dirette al source di Unreal Engine; comporta quindi il mantenimento di un engine fork. ¹¹

VaRest è stato archiviato nel novembre 2024 e non dovrebbe essere scelto come nuova dipendenza. Per eventuali API web sono preferibili i moduli HTTP e JSON nativi di Unreal o una libreria mantenuta. ¹²

Decisione raccomandata

La baseline è:

Gameplay core	→ C++
Contenuti	→ Blueprint + Primary Data Assets
Classificazione	→ Gameplay Tags
UI	→ UMG, eventualmente MVVM
Networking	→ Replicazione Unreal e RPC
Test	→ Automation Framework + Gauntlet
Mod dati	→ JSON + registry
Mod contenuti	→ Pak/chunk
Mod scripting	→ Lua sandboxata
C#	→ Spike opzionale, non dipendenza core

Struttura del progetto

```
RefactorTactics/  
├─ Config/  
├─ Content/  
│   ├─ Characters/  
│   ├─ Abilities/  
│   ├─ Maps/  
│   ├─ Tiles/  
│   └─ UI/  
├─ Developer/  
├─ Plugins/  
│   ├─ RefactorTacticsCore/  
│   ├─ RefactorTacticsMap/  
│   ├─ RefactorTacticsCombat/  
│   ├─ RefactorTacticsMod/  
│   └─ ThirdParty/
```



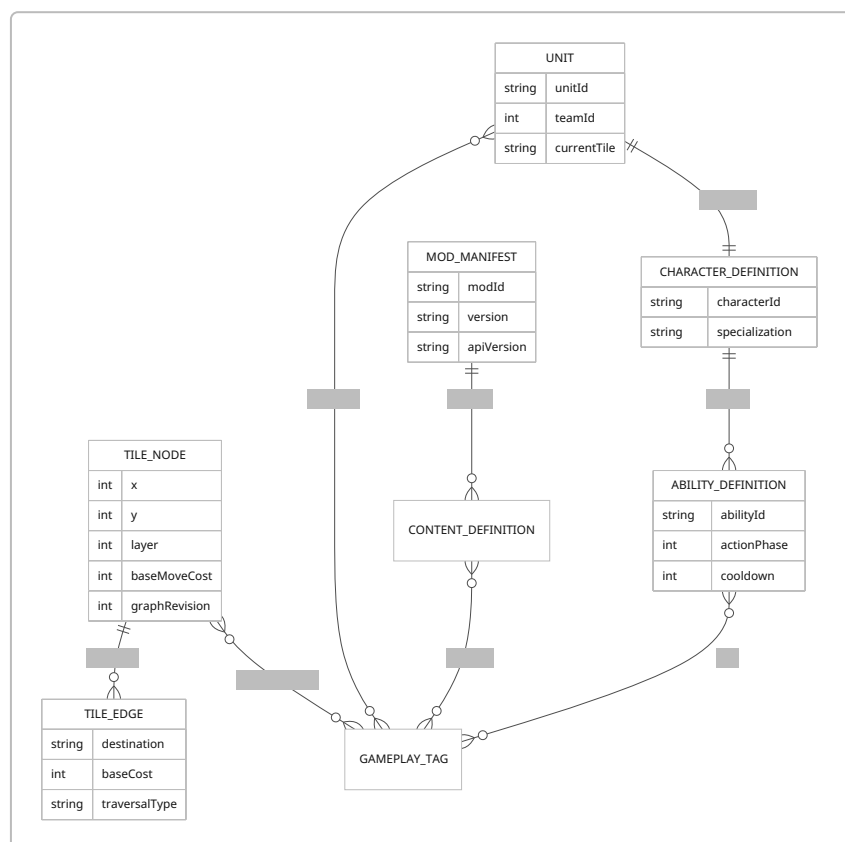
```

└─ Source/
  └─ RefactorTactics/
    └─ RefactorTacticsEditor/
      └─ RefactorTacticsServer/
└─ Mods/
└─ Scripts/
└─ RefactorTactics.uproject

```

Per le prime lezioni può essere utilizzato un solo modulo. La separazione in plugin va introdotta soltanto quando i confini diventano stabili. I plugin Unreal possono contenere codice, contenuti ed estensioni dell'Editor. 13

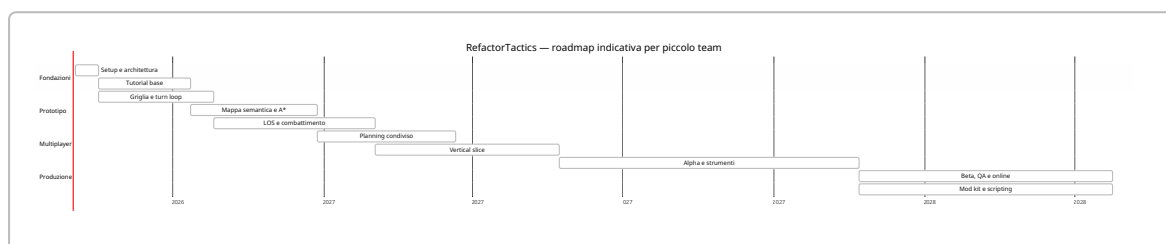
Modello dati principale



Roadmap dal tutorial al prodotto

Roadmap Gantt

Timeline semplificata



Le date del diagramma sono illustrative. I range seguenti sono le vere stime operative.

Milestone e criteri di successo

Milestone	Stima	Deliverable	Dipendenze	Criteri di successo
Fondazioni	1-2 settimane	Repository, engine, IDE, build e convenzioni	Nessuna	Progetto compilabile su un secondo PC
Tutorial base	6-10 settimane	Camera, input, tile selection, primo Actor C++	Fondazioni	Piccola scena giocabile e versionata
Turn loop locale	4-8 settimane	Planning, conferma e risoluzione locale	Tutorial base	Match di almeno cinque turni senza soft-lock
Griglia multilivello	6-10 settimane	Layer, archi verticali e debug tool	Griglia base	Percorsi corretti tra almeno tre piani
Pathfinding semantico	5-9 settimane	A*, provider, cache e test	Griglia multilivello	Suite di test completa e budget prestazionale rispettato
Combat sandbox	10-16 settimane	LOS, cover, danni, stati e ambiente	Pathfinding	Scenario tattico completo con quattro unità
Personaggi modulari	8-14 settimane	Kit, specializzazioni, augment e gadget	Combat sandbox	Almeno quattro eroi e due build significative ciascuno
Planning multiplayer	8-14 settimane	Server authority, intent alleati, commit	Turn loop	Due team giocano senza leak del planning
Vertical slice	12-20 settimane	Una mappa, otto eroi, UI, audio, onboarding	Tutti i sistemi P0	Sessione completa di 20-30 minuti
Alpha	20-36 settimane	Contenuti, tool, matchmaking base, telemetria	Vertical slice	Feature complete e bilanciabile

Milestone	Stima	Deliverable	Dipendenze	Criteri di successo
Beta	16–28 settimane	Ottimizzazione, accessibilità, sicurezza e QA	Alpha	Target prestazioni e rete rispettati
Mod kit	16–28 settimane	Editor, validator, JSON, Lua e packaging	API gameplay stabile	Mod di esempio installabile senza ricompilare il gioco
Release candidate	6–12 settimane	Stabilizzazione, documentazione e distribuzione	Beta e mod kit	Zero blocker e procedure di rollback pronte

Stime complessive

Scenario	Impegno	Intervallo plausibile
Sviluppatore singolo	10–15 ore alla settimana	24–42 mesi
Sviluppatore singolo full-time	35–40 ore alla settimana	18–30 mesi
Team piccolo	3–5 persone full-time	14–24 mesi
Team con produzione contenuti separata	6–10 persone	12–20 mesi

Le stime non includono certificazione console, localizzazione estesa, infrastruttura live-service, supporto clienti o una grande campagna narrativa.

Gate del vertical slice

Il vertical slice deve contenere soltanto:

- una modalità;
- una mappa multilivello;
- otto personaggi;
- due specializzazioni per personaggio;
- un set limitato di terreni;
- planning privato tra alleati;
- un tutorial di onboarding;
- bot di prova;
- replay tecnico o event log;
- nessun editor mod pubblico.

Il progetto non dovrebbe entrare in produzione estesa finché il vertical slice non dimostra:

Area	Target iniziale
Durata partita	20–35 minuti
Comprensibilità del planning	Almeno l'80% dei tester interpreta correttamente il piano alleato
Stabilità interna	Almeno il 95% delle sessioni termina senza crash

Area	Target iniziale
Tempo di planning	La maggioranza dei turni si conclude entro il timer
Pathfinding	Nessuna destinazione raggiungibile dichiarata irraggiungibile
Privacy	Nessun intent avversario in memoria applicativa, actor o log del client
Bilanciamento	Nessuna build scelta in oltre il 60% dei casi per pura superiorità
Mappa	Almeno tre interazioni ambientali usate spontaneamente per partita

Sequenza delle lezioni

Ambiente di sviluppo

La documentazione ufficiale raccomanda per Unreal Engine un sistema Windows 10 o 11 a 64 bit, CPU quad-core, 32 GB di RAM e GPU compatibile DirectX 11 o 12 con almeno 8 GB di memoria grafica. Per compilazioni frequenti e mappe complesse è prudente utilizzare 64 GB di RAM, SSD NVMe e una GPU con 12 GB di VRAM, ma questi ultimi sono consigli pratici del progetto, non requisiti ufficiali. ¹⁴

Configurazione Windows:

1. Installare Epic Games Launcher.
2. Installare Unreal Engine 5.8.1.
3. Installare Visual Studio 2022.
4. Selezionare il workload **Game development with C++**.
5. Installare Windows SDK e Visual Studio Tools for Unreal Engine.
6. Installare Git e Git LFS.
7. Creare una cartella senza spazi eccessivi, per esempio `D:\Dev\RefactorTactics`.
8. Creare un progetto `Games → Blank → C++`.
9. Compilare la configurazione `Development Editor | Win64`.
10. Creare il primo commit.

Epic documenta sia l'installazione del motore sia la configurazione di Visual Studio per i progetti C++. ¹⁵

Hardware consigliato per il percorso:

Componente	Minimo didattico	Consigliato
CPU	4 core moderni	8-16 core
RAM	32 GB	64 GB
GPU	8 GB VRAM	12 GB VRAM
Disco	300 GB liberi	1 TB NVMe libero
Sistema	Windows 10/11 64 bit	Windows 11
IDE	Visual Studio 2022	Visual Studio 2022 o Rider

`.gitignore` iniziale:

```
Binaries/  
DerivedDataCache/  
Intermediate/  
Saved/  
.vs/  
*.sln  
.vscode/
```

Cartelle da versionare:

```
Config/  
Content/  
Plugins/  
Source/  
RefactorTactics.uproject
```

File binari Unreal come `.uasset` e `.umap` devono essere gestiti con Git LFS:

```
git lfs install  
git lfs track "*.uasset"  
git lfs track "*.umap"  
git add .gitattributes
```

Ponte mentale da C# a C++ Unreal

C# / Unity	Unreal
<code>GameObject</code>	<code>AActor</code>
<code>MonoBehaviour</code>	<code>UActorComponent</code> o <code>AActor</code>
Prefab	Blueprint Class
<code>ScriptableObject</code>	<code>UDataAsset</code> / <code>UPrimaryDataAsset</code>
Inspector field	<code>UPROPERTY(EditAnywhere)</code>
UnityEvent	Delegate Unreal
Coroutine	Timer, latent action o task
<code>List<T></code>	<code>TArray<T></code>
<code>Dictionary<K,V></code>	<code>TMap<K,V></code>
<code>HashSet<T></code>	<code>TSet<T></code>
<code>null</code> managed	<code>nullptr</code> e validità <code>UObject</code>

C# / Unity	Unreal
Package assembly	Module Unreal

Primo esempio copiabile:

```
#pragma once

#include "CoreMinimal.h"
#include "GameFramework/Actor.h"
#include "TutorialTile.generated.h"

UCLASS()
class REFACTORTACTICS_API ATutorialTile : public AActor
{
    GENERATED_BODY()

public:
    ATutorialTile();

    UPROPERTY(EditAnywhere, BlueprintReadWrite, Category="Tile")
    int32 MovementCost = 10;

    UPROPERTY(EditAnywhere, BlueprintReadWrite, Category="Tile")
    bool bBlocksMovement = false;

    UFUNCTION(BlueprintCallable, Category="Tile")
    bool CanEnter() const;
};
```

```
#include "TutorialTile.h"

ATutorialTile::ATutorialTile()
{
    PrimaryActorTick.bCanEverTick = false;
}

bool ATutorialTile::CanEnter() const
{
    return !bBlocksMovement;
}
```

Passi:

1. Creare `TutorialTile.h`.
2. Creare `TutorialTile.cpp`.
3. Compilare.
4. Nell'Editor creare `BP_TutorialTile` come child della classe.
5. Aggiungere una Static Mesh.

6. Modificare `MovementCost` dal pannello Details.
7. Posizionare più tile nella scena.

Risultato: il codice definisce regole e proprietà, mentre Blueprint definisce aspetto e configurazione.

Piano didattico completo

Lezione	Obiettivo	Output
Ambiente	Installare engine, IDE, Git LFS e compilare	Build pulita
Modello Unreal	Imparare Actor, Component, UObject e reflection	Actor C++ modificabile da Blueprint
Camera tattica	Pan, zoom, rotazione e selezione	Camera controllabile
Enhanced Input	Separare input da logica	Mapping context funzionante
Tile picking	Raycast e coordinate di griglia	Tile evidenziata al passaggio
Griglia base	Generare nodi e visualizzarli	Griglia selezionabile
Griglia multilivello	Layer e archi verticali	Percorso tra piani
Tile semantiche	Tag, costi, hazard e Data Assets	Acqua, fuoco e ghiaccio
A* base	Open set, costo ed euristica	Percorso minimo
Cost provider	Modificatori per unità e ambiente	Percorsi diversi per personaggio
LOS	Visibilità e cover separate	Preview di tiro
Turn orchestrator	Planning, commit e resolve	Turno locale completo
Personaggi	Kit, specializzazioni e augment	Due build per eroe
UI planning	Ghost, spline, AoE e intent	Piano comprensibile
Multiplayer	Server, RPC e PlayerState	Match a due client
Privacy team	Inoltro intent solo agli alleati	Test leak negativo
Test	Automation, PIE, emulazione rete	Suite smoke
Data mod	Manifest e JSON registry	Nuovo terreno esterno
Editor mod	Tool, validazione e packaging	Mod creata dall'Editor
Lua sandbox	Script e capability	Trigger mod sicuro

Lezione sulla camera tattica

Creare un `Pawn` con:

```

Scene Root
├─ Spring Arm
└─ Camera

```

Input:

Azione	Binding
IA_CameraMove	WASD
IA_CameraZoom	Rotella mouse
IA_CameraRotate	Q/E
IA_Select	Tasto sinistro
IA_Cancel	Tasto destro

Logica Blueprint iniziale:

```

IA_CameraMove
→ Get Control Rotation
→ Forward/Right Vector
→ Add Actor World Offset

```

In questa fase è corretto usare Blueprint. La conversione in C++ serve soltanto quando la logica diventa condivisa o richiede test automatici.

Lezione sulla griglia multilivello

Struttura minima:

```

USTRUCT(BlueprintType)
struct FTileEdge
{
    GENERATED_BODY()

    UPROPERTY(EditAnywhere, BlueprintReadWrite)
    FTileId To;

    UPROPERTY(EditAnywhere, BlueprintReadWrite)
    int32 BaseCost = 10;

    UPROPERTY(EditAnywhere, BlueprintReadWrite)
    FGameplayTagContainer Requirements;
};

USTRUCT(BlueprintType)
struct FTileNode

```



```

{
    GENERATED_BODY()

    UPROPERTY(EditAnywhere, BlueprintReadWrite)
    FTileId Id;

    UPROPERTY(EditAnywhere, BlueprintReadWrite)
    int32 HeightSteps = 0;

    UPROPERTY(EditAnywhere, BlueprintReadWrite)
    int32 BaseMoveCost = 10;

    UPROPERTY(EditAnywhere, BlueprintReadWrite)
    FGameplayTagContainer Tags;

    UPROPERTY(EditAnywhere, BlueprintReadWrite)
    TArray<FTileEdge> Edges;
};

```

Passi:

1. Generare una griglia sul layer zero.
2. Aggiungere una seconda griglia sul layer uno.
3. Collegare manualmente due celle con un arco `Traversal.Stairs`.
4. Aggiungere il requisito `Movement.Climb`.
5. Assegnare a un personaggio il tag richiesto.
6. Verificare che il personaggio possa salire.
7. Rimuovere il tag.
8. Verificare che il percorso non utilizzi più la scala.
9. Aggiungere un ascensore con requisito `State.Powered`.
10. Modificare lo stato durante il turno e invalidare la revisione del grafo.

Lezione su A*

Versione iniziale, volutamente semplice:

```

int32 Heuristic(const FTileId& A, const FTileId& B)
{
    return 10 * (
        FMath::Abs(A.X - B.X) +
        FMath::Abs(A.Y - B.Y) +
        FMath::Abs(A.Layer - B.Layer)
    );
}

```

Costo di un vicino:

```

int32 CalculateTraversalCost(
    const FTileNode& Current,

```

```

    const FTileEdge& Edge,
    const FTileNode& Next)
{
    return Edge.BaseCost + Next.BaseMoveCost;
}

```

Prima estensione:

```

int32 CalculateTraversalCost(
    const FTileNode& Current,
    const FTileEdge& Edge,
    const FTileNode& Next,
    const FUnitTraversalContext& Unit,
    const TArray<ITraversalCostProvider*>& Providers)
{
    int32 Cost = Edge.BaseCost + Next.BaseMoveCost;

    for (const ITraversalCostProvider* Provider : Providers)
    {
        if (!Provider->CanTraverse(Current, Next, Unit))
        {
            return MAX_int32;
        }

        Cost += Provider->GetAdditionalCost(
            Current,
            Next,
            Unit
        );
    }

    return FMath::Max(1, Cost);
}

```

Test necessari:

```

Percorso su terreno normale
Percorso attorno a ostacolo
Percorso tra due layer
Scala senza requisito
Scala con requisito
Acqua per umano
Acqua per unità anfibia
Cella resa invalida durante la partita
Start uguale a goal
Goal irraggiungibile

```

Lezione sul turn orchestrator

Stati:

```
UENUM(BlueprintType)
enum class ETurnPhase : uint8
{
    Planning,
    Locked,
    ResolvingMovement,
    ResolvingReactions,
    ResolvingActions,
    ResolvingEnvironment,
    Cleanup
};
```

Il server conserva:

```
USTRUCT()
struct FCommittedTurnIntent
{
    GENERATED_BODY()

    UPROPERTY()
    int32 PlayerId = INDEX_NONE;

    UPROPERTY()
    int32 UnitId = INDEX_NONE;

    UPROPERTY()
    TArray<FTileId> Path;

    UPROPERTY()
    FName AbilityId;

    UPROPERTY()
    FTileId TargetTile;

    UPROPERTY()
    int32 Sequence = 0;
};
```

Passi:

1. Avviare **Planning**.
2. Accettare preview non definitiva.
3. Accettare il commit.
4. Validare il piano.
5. Impedire modifiche dopo il lock.

6. Ordinare gli eventi per fase e priorità.
7. Risolvere il movimento.
8. Risolvere azioni e ambiente.
9. Salvare l'event log.
10. Avviare il turno successivo.

Lezione sul planning multiplayer

L'architettura client-server e il sistema di replicazione di Unreal sono documentati come base del multiplayer; il server deve essere l'autorità sullo stato condiviso. 16

Struttura dell'intent:

```
USTRUCT(BlueprintType)
struct FPlanningIntent
{
    GENERATED_BODY()

    UPROPERTY(BlueprintReadWrite)
    int32 Sequence = 0;

    UPROPERTY(BlueprintReadWrite)
    FName AbilityId;

    UPROPERTY(BlueprintReadWrite)
    TArray<FTileId> PlannedPath;

    UPROPERTY(BlueprintReadWrite)
    FTileId TargetTile;

    UPROPERTY(BlueprintReadWrite)
    bool bCommitted = false;
};
```

Passi:

1. Aggiungere `TeamId` al `PlayerState`.
2. Impostarlo esclusivamente sul server.
3. Costruire la preview localmente.
4. Inviare l'intent al server con RPC non affidabile.
5. Verificare ownership e team.
6. Inoltrare soltanto ai `PlayerController` alleati.
7. Disegnare spline e ghost sul client alleato.
8. Eliminare la preview quando cambia turno.
9. Utilizzare un RPC affidabile per il commit.
10. Ispezionare il client nemico per verificare che il payload non esista.

Il file nel kit contiene un metodo `IsSameTeam` volutamente provvisorio. Deve essere sostituito da un vero controllo su `TeamId` prima di qualsiasi test online.

Lezione sulla UI

Widget principali:

```
WBP_TacticalHUD
├─ WBP_TurnTimer
├─ WBP_AbilityBar
├─ WBP_TeamPlanningStatus
├─ WBP_IntentLabel
├─ WBP_ConflictWarning
└─ WBP_TileInspector
```

Visualizzazioni nel mondo:

```
BP_PlannedPathSpline
BP_AllyGhost
BP_TargetAreaPreview
BP_FacingArrow
BP_ContextPing
```

Regole di accessibilità:

- non utilizzare solo il colore;
- assegnare anche pattern, icone o stili di linea;
- consentire dimensionamento della UI;
- mostrare testo per i conflitti;
- supportare rimappatura degli input;
- limitare il numero di preview simultanee;
- attenuare i piani non selezionati;
- permettere di isolare il piano di un singolo alleato.

Lezione sui test

Unreal offre Automation Test Framework per test automatici e Gauntlet per orchestrare sessioni più complesse, incluse configurazioni con server e più client. PIE consente di testare più giocatori dall'Editor, mentre la network emulation permette di simulare latenza e perdita di pacchetti. ¹⁷

Matrice minima:

Test	Ambiente
A* e cost provider	Automation Test
Serialize/deserialize mod	Automation Test
Turn resolve deterministico	Automation Test
Due client e server	PIE
Due squadre e planning privato	PIE o Gauntlet

Test	Ambiente
100–200 ms latenza	Network emulation
2–5% packet loss	Network emulation
Disconnect durante planning	Gauntlet
Manifest mod differente	Packaged build
Dedicated server senza rendering	Packaged server

Esecuzione indicativa:

```
UnrealEditor-Cmd.exe RefactorTactics.uproject
-ExecCmds="Automation RunTests RefactorTactics"
-unattended
-TestExit="Automation Test Queue Empty"
```

I parametri precisi possono cambiare con la versione del motore; la documentazione ufficiale descrive i comandi e il report dei test. ¹⁸

Lezione sulle mod

Ordine corretto:

```
Registry interno
→ Data Assets
→ JSON schema
→ Data mod locali
→ Content package
→ Editor mod
→ Lua sandbox
→ Catalogo pubblico
```

Il primo obiettivo non è permettere qualunque modifica, ma registrare nuovi dati senza modificare il core:

```
USTRUCT(BlueprintType)
struct FTerrainDefinition
{
    GENERATED_BODY()

    UPROPERTY(EditAnywhere, BlueprintReadOnly)
    FName Id;

    UPROPERTY(EditAnywhere, BlueprintReadOnly)
    FText DisplayName;
```

```

UPROPERTY(EditAnywhere, BlueprintReadOnly)
FGameplayTagContainer Tags;

UPROPERTY(EditAnywhere, BlueprintReadOnly)
int32 BaseMoveCost = 10;

UPROPERTY(EditAnywhere, BlueprintReadOnly)
bool bBlocksMovement = false;

UPROPERTY(EditAnywhere, BlueprintReadOnly)
bool bBlocksVision = false;
};

```

Validator:

```

ID univoco
Versione schema valida
Tag registrati
Dipendenze presenti
Nessun costo negativo
Nessun riferimento circolare proibito
Asset consentiti
Script entro limiti
Hash calcolato

```

Game Features e Modular Gameplay possono essere studiati per modularizzare contenuti, ma la documentazione li presenta ancora con avvertenze relative allo stato Beta; non dovrebbero diventare automaticamente la base del sistema mod senza una spike. ¹⁹

Rischi, qualità e learning curve

Rischi tecnici

Rischio	Probabilità	Impatto	Mitigazione
Scope eccessivo	Alta	Alto	Un'unica modalità, una mappa e otto eroi nel vertical slice
Troppe combinazioni	Alta	Alto	Gameplay Tags, contratti, budget effetti e test automatici
Leak del planning	Media	Critico	Payload team-only, packet inspection e test con client nemico
Dipendenza da plugin C#	Alta	Medio	Spike di massimo due settimane e core completamente nativo
Desync della risoluzione	Media	Alto	Server authority, costi interi, RNG centralizzato ed event log

Rischio	Probabilità	Impatto	Mitigazione
Pathfinding lento	Media	Alto	Profiling, cache per revisione e invalidazione regionale
LOS incoerente	Media	Alto	Query tattica condivisa da preview e server
Mod malevole	Media	Critico	Sandbox, capability, timeout e divieto di native code pubblico
API mod instabile	Alta	Alto	Mod support dopo beta tecnica e versionamento semantico
Tooling insufficiente	Media	Alto	Debug overlay e validator sviluppati insieme ai sistemi
Bilanciamento ingestibile	Alta	Alto	Slot, incompatibilità e telemetria sulle build
Rete fragile	Media	Alto	PIE, emulazione, Gauntlet e dedicated server test

Learning curve per uno sviluppatore C

Con un impegno di 10–15 ore settimanali:

Area	Tempo indicativo
Editor e Blueprint	2–4 settimane
Actor, Component e UObject	2–4 settimane
C++ Unreal quotidiano	4–8 settimane
Memory model e reflection	4–8 settimane
UMG e input	3–6 settimane
Pathfinding e test	6–12 settimane
Networking e lifecycle	8–14 settimane
Tool per l'Editor	10–16 settimane
Packaging e mod	12–20 settimane

Queste fasi si sovrappongono. Non è necessario “imparare tutto il C++” prima di iniziare: è più efficace imparare un sottoinsieme Unreal mirato:

```
UCLASS, USTRUCT, UENUM
UPROPERTY, UFUNCTION
AActor, UObject, UActorComponent
TArray, TMap, TSet
TObjectPtr e riferimenti deboli
Delegate
ModuleRules
```


Regole di qualità

Il core tattico deve essere:

- deterministico sul server;
- indipendente dagli effetti visivi;
- interrogabile attraverso API pure dove possibile;
- serializzabile in event log;
- testabile senza avviare una mappa completa;
- data-driven;
- compatibile con revisioni del grafo;
- protetto da input non validi;
- misurabile tramite profiling.

Criteri prestazionali iniziali

I valori seguenti sono target interni da misurare e adattare:

Sistema	Target prototipo
A* singolo	p95 inferiore a 2 ms sulla mappa target
Batch preview	Inferiore a 8 ms per frame, con throttling
Aggiornamento intent	5–10 Hz durante modifica
Payload intent	Idealmente sotto 2–4 KB
Turn resolve	Inferiore a 100 ms lato server
LOS batch	Cache per turno e invalidazione mirata
Script mod	Budget per callback e interruzione automatica
Frame rate PC	60 FPS su hardware target
Server dedicato	Nessuna dipendenza da rendering

Questi non sono standard esterni: sono budget iniziali del progetto. Devono essere validati con una mappa rappresentativa, non con una scena vuota.

Definition of done per il supporto mod

Il supporto mod può essere considerato pronto quando:

1. una mod aggiunge un terreno senza ricompilare;
2. una mod aggiunge una nuova abilità usando soltanto dati approvati;
3. una mod scriptata non può leggere filesystem o rete liberamente;
4. uno script in loop viene terminato;
5. una mod con API incompatibile viene rifiutata con un errore leggibile;
6. server e client confrontano gli hash;

7. una mod può essere disabilitata senza corrompere i salvataggi;
8. il validator produce un report;
9. il packaging genera un artefatto separato;
10. la documentazione include almeno una mod completa di esempio.

Artefatti stampabili e starter pack

Il kit prodotto include:

File	Scopo
<code>RefactorTactics_PRD_Roadmap_Tutorial_A4.pdf</code>	Documento principale pronto per la stampa
<code>RefactorTactics_PRD_Roadmap_Tutorial_A4.html</code>	Versione modificabile e stampabile da browser
<code>architettura_refactortactics.png</code>	Diagramma a 300 DPI
<code>architettura_refactortactics.svg</code>	Diagramma vettoriale
<code>roadmap_gantt.png</code>	Roadmap a 300 DPI
<code>roadmap_gantt.svg</code>	Roadmap vettoriale
<code>core_loop_turno.png</code>	Core loop a 300 DPI
<code>core_loop_turno.svg</code>	Core loop vettoriale
<code>TileTypes.h</code>	Tipi copiabili per tile e archi
<code>SemanticPathfinder.h/.cpp</code>	A* didattico
<code>PlanningPlayerController.h/.cpp</code>	Esempio RPC per planning alleato
<code>RefactorTactics.Build.cs</code>	Dipendenze iniziali del modulo
<code>mod.json</code>	Manifest mod
<code>terrain_electrified_water.json</code>	Esempio data mod
<code>electrified_water.lua</code>	Esempio di script sandboxato

Download separati dei diagrammi:

- [Architettura PNG](#)
- [Architettura SVG](#)
- [Roadmap PNG](#)
- [Roadmap SVG](#)
- [Core loop PNG](#)
- [Core loop SVG](#)

La versione PDF contiene anche la bibliografia con i collegamenti ufficiali a installazione, requisiti hardware, Visual Studio, framework di gameplay, networking, relevancy, Gameplay Tags, Data Assets, Asset Manager, plugin, packaging, test, UnrealCLR, UnLua, AngelScript e letteratura sul pathfinding.

- 1 <https://forums.unrealengine.com/t/unreal-engine-5-8-released/2729274>
<https://forums.unrealengine.com/t/unreal-engine-5-8-released/2729274>
- 2 <https://ui.adsabs.harvard.edu/abs/1968IJSSC...4..100H/abstract>
<https://ui.adsabs.harvard.edu/abs/1968IJSSC...4..100H/abstract>
- 3 <https://www.cs.cmu.edu/~ggordon/likhachev-et.al.anytime-dstar.pdf>
<https://www.cs.cmu.edu/~ggordon/likhachev-et.al.anytime-dstar.pdf>
- 4 **Actor Relevancy in Unreal Engine**
https://dev.epicgames.com/documentation/unreal-engine/actor-relevancy-in-unreal-engine?utm_source=chatgpt.com
- 5 <https://dev.epicgames.com/documentation/unreal-engine/experimental-features>
<https://dev.epicgames.com/documentation/unreal-engine/experimental-features>
- 6 **Using Gameplay Tags in Unreal Engine**
https://dev.epicgames.com/documentation/unreal-engine/using-gameplay-tags-in-unreal-engine?utm_source=chatgpt.com
- 7 <https://dev.epicgames.com/documentation/unreal-engine/gameplay-framework-in-unreal-engine>
<https://dev.epicgames.com/documentation/unreal-engine/gameplay-framework-in-unreal-engine>
- 8 <https://github.com/nxrighthere/UnrealCLR>
<https://github.com/nxrighthere/UnrealCLR>
- 9 **MonoUE | Mono for Unreal Engine is a plugin for Unreal ...**
https://mono-ue.github.io/?utm_source=chatgpt.com
- 10 <https://github.com/tencent/unlua>
<https://github.com/tencent/unlua>
- 11 **Unreal Engine Angelscript - Hazelight**
https://angelscript.hazelight.se/?utm_source=chatgpt.com
- 12 <https://github.com/ufna/VaRest>
<https://github.com/ufna/VaRest>
- 13 <https://dev.epicgames.com/documentation/unreal-engine/plugins-in-unreal-engine>
<https://dev.epicgames.com/documentation/unreal-engine/plugins-in-unreal-engine>
- 14 <https://dev.epicgames.com/documentation/unreal-engine/hardware-and-software-specifications-for-unreal-engine>
<https://dev.epicgames.com/documentation/unreal-engine/hardware-and-software-specifications-for-unreal-engine>
- 15 <https://dev.epicgames.com/documentation/unreal-engine/install-unreal-engine>
<https://dev.epicgames.com/documentation/unreal-engine/install-unreal-engine>
- 16 **Networking and Multiplayer in Unreal Engine**
https://dev.epicgames.com/documentation/unreal-engine/networking-and-multiplayer-in-unreal-engine?utm_source=chatgpt.com
- 17 <https://dev.epicgames.com/documentation/unreal-engine/automation-test-framework-in-unreal-engine>
<https://dev.epicgames.com/documentation/unreal-engine/automation-test-framework-in-unreal-engine>
- 18 <https://dev.epicgames.com/documentation/unreal-engine/running-gauntlet-tests-in-unreal-engine>
<https://dev.epicgames.com/documentation/unreal-engine/running-gauntlet-tests-in-unreal-engine>
- 19 <https://dev.epicgames.com/documentation/unreal-engine/game-features-and-modular-gameplay-in-unreal-engine>
<https://dev.epicgames.com/documentation/unreal-engine/game-features-and-modular-gameplay-in-unreal-engine>